Introduction to Computer Science and Programming

Labo 16 – C++ Dynamic allocation and dynamic arrays

Laboratory 16, April 09, 2026

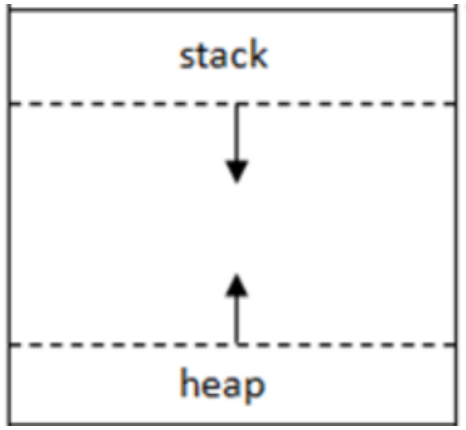
Filippo Ricca & Muhammad Mohsin

DIBRIS, University of Genova

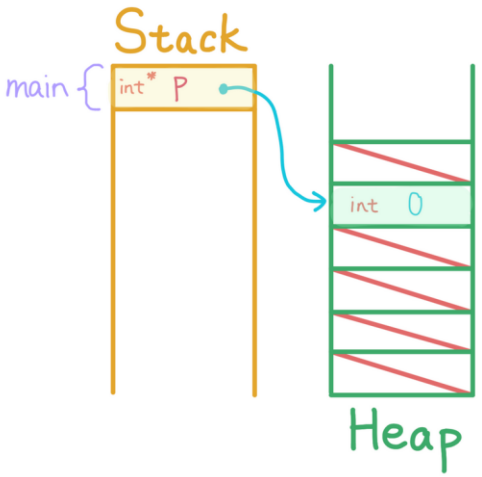
Today…

Today's lab is dedicated to **dynamic allocation and dynamic arrays**

- Exercise 1: **Resizing Dynamic Array**
- Exercise 2: **Dynamic Array Reversal**
- Exercise 3: **Greater than Average Number**
- Exercise 4: **Dynamic Odd Number**
- Exercise 5: **Dynamic 2D Array Allocation**
- Exercise 6: **Memory Leak in dynamic allocation**



All the C++ program must be delivered on Aulaweb!




```
int main() {
  int *p = new int;
  Cube *c1 = new Cube();
  Cube *c2 = c1;
  c2->setLength( 10 );
  return 0;
}
```

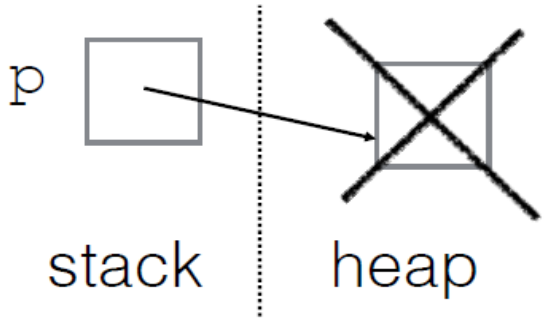
Dynamic Allocation

Dynamic memory allocation is the process of allocating memory at the runtime of a program.

In **dynamic memory allocation**, we **allocate** and then **deallocate** memory dynamically using the **new and delete operators** respectively.

```
p = new type;
p = new type [number_of_elements];

Delete [] p;
```



Heap Memory

This allocation performed in the **Heap memory** of the program instead of the **stack memory**.

Why?

- When you are not sure about the size of the array you need.
- Implementing data structures such as linked list, trees, etc.
- In complex programs that require efficient memory management.

Example.

```
int* arr = new int[10];   // Dynamic array of 10 integers
```

Declaration of dynamic array

```
dataType* arrayName = new dataType[size];
```

```
int size = 5;
int* dynamicArray = new int[size]; // Allocates array of 5 integers
```

Remember that dynamic array could be of any size defined by the user, for instance in above example, size=5

Exercise 1: Resizing a Dynamic Array

Write a C++ program in which the user dynamically allocates an array of **‘n’ integers.** When the array becomes full, create a new array with **2 × current capacity,** copy all existing elements into it, delete the old memory, and continue using the new array.

- **Example:** If capacity = 4 and array is full, create a new array of capacity = 8.
- **Hint:** Use dynamic memory allocation
- (new / delete[]) and copy elements before freeing the old array.

Exercise 2: Dynamic Array Reversal

Write a C++ program that dynamically allocates an integer array of size **'n'**, initializes it with values 1 to n, then reverses the array (**without using extra memory**)

For Example:

Enter array size: 5

Original array: 1 3 5 7 9

Reversed array: 9 7 5 3 1

Don't forget to deallocate the dynamic memory to avoid memory leaks

Exercise 3: Dynamic Odd Number

Write a C++ program in which user dynamically allocate an array of **'n'** integers. Input **'n'** values and create another array with only odd numbers.

Example: Input: [2, 3, 5, 8, 11] → Output: [3, 5, 11]

Hint: Use two passes — first to count, second to copy.

Don't forget to deallocate the dynamic memory to avoid memory leaks

Exercise 4: Greater than average number

Write a C++ program in which Allocate **float array dynamically,** find average of the numbers present in arrays.

Then print only the values greater than average value.

Example:

[2.5, 3.5, 6.0, 4.3, 6.7] → Avg = 4.6 → Output: 6.0, 6.7

Don’t forget to deallocate the dynamic memory to avoid memory leaks

Exercise 5: Dynamic 2D Array Allocation

Write a C++ program in which the user dynamically allocates a **2D array of floats** with dimensions **m × n**. Initialize the array with USER INPUT values and print the complete matrix.

Example:

Input: m = 3, n = 3

Output:

[1.2 3.5 4.1]

[5.6 7.3 0.9]

[8.1 2.7 3.3]

Hint: Use new to allocate rows first, then allocate columns for each row.

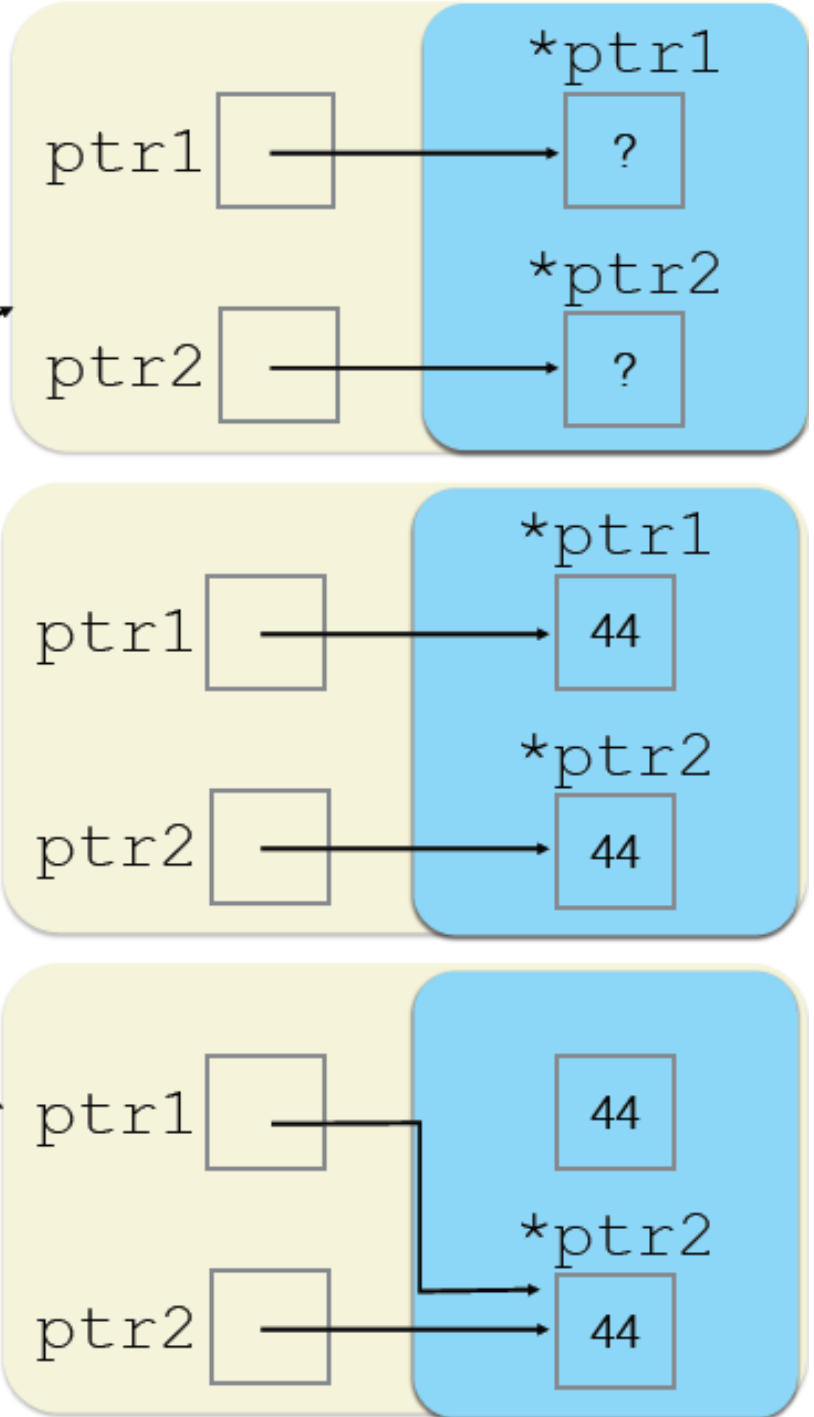
Memory Leak example

```
int *ptr1 = new int;
int *ptr2 = new int;

*ptr2 = 44;
*ptr1 = *ptr2;

ptr1 = ptr2;

delete ptr2;
ptr1=nullptr;
ptr2=nullptr;
```



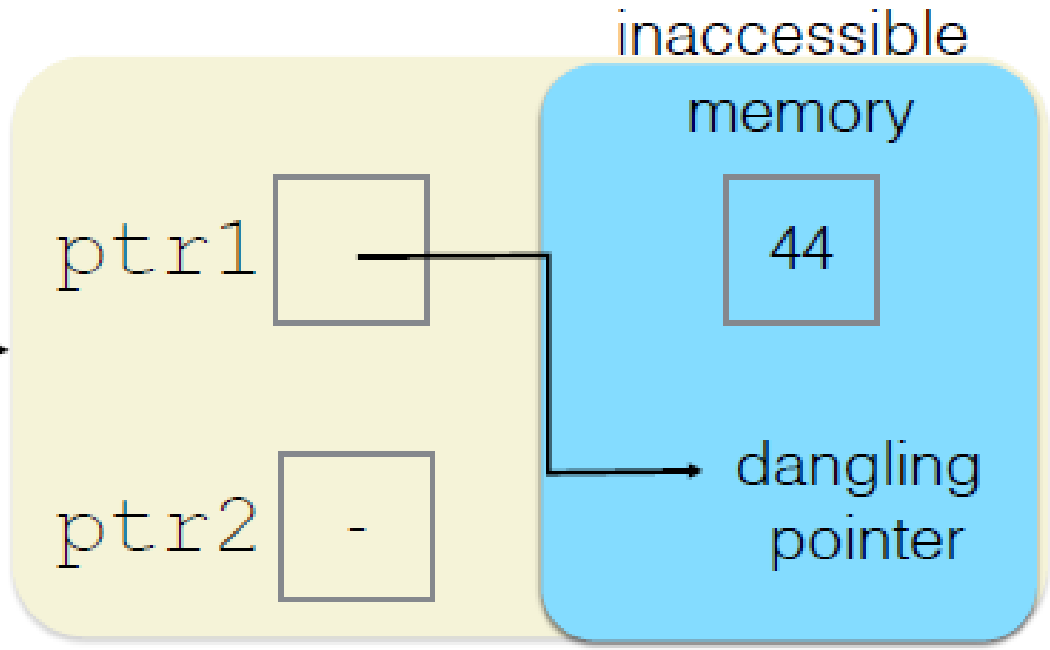
Memory Leak example

```
int *ptr1 = new int;
int *ptr2 = new int;


*ptr2 = 44;
*ptr1 = *ptr2;


ptr1 = ptr2;


delete ptr2;
ptr2 = nullptr;
```



Exercise 6: Memory Leak in dynamic allocation

Write a C++ program that dynamically allocates memory for an integer **array (arr1) of size 100** then reassigns the same pointer to a new **array (arr2) of size 50.**

Next, the program correctly deletes it using **delete[],** but the original arr2 array is never deallocated.

- Your task is to run this program, identify where the **memory leak** occurs.
- Explain why it happens.
- Modify the program to fix the leak by ensuring that all dynamically allocated memory is properly released before the pointer is reassigned.